

Ans Sheet: CS & AI Master Exam - Aarav Patel (Roll: CS-2026-001)Q1: Backpropagation in Deep Neural Networks

Forward pass computes network activations $z = w^*x + b$ and prediction $\hat{y}$. Loss L is computed against target y using loss function.

Backward pass computes partial derivatives of loss wrt weights using chain rule:

$$dL/dw = (dL/da) * (da/dz) * (dz/dw)$$

Where $dz/dw = x$. Gradient descent updates weights: $w = w - \eta * (dL/dw)$.

Error gradients propagate backward layer by layer from output to input.

Q2: Time & Space Complexity: QuickSort vs MergeSort

1. MergeSort:

- Divide & Conquer algorithm splitting array into sub-arrays.
- Average Time: $O(N \log N)$
- Worst Time: $O(N \log N)$ (guaranteed balanced splitting)
- Space: $O(N)$ auxiliary array space for merging sub-arrays.

2. QuickSort:

- Partitioning algorithm around chosen pivot element.
- Average Time: $O(N \log N)$
- Worst Time: $O(N^2)$ occurs when array is already sorted and worst pivot (min/max) is selected.
- Space: $O(\log N)$ for call stack recursion (in-place partitioning).

Q3: ACID Properties in Relational Databases (RDBMS)

- *Atomicity: Transactions are 'all or nothing'. If any operation fails, entire transaction rolls back to initial state.*
- *Consistency: Database transitions from one valid state to another, maintaining all schema constraints and foreign keys.*
- *Isolation: Concurrent transactions execute without inter-thread interference (achieved via locking or MVCC).*
- *Durability: Once committed, data changes are written to non-volatile storage (WAL log) and persist across system crashes.*

Q4: Core Object-Oriented Programming (OOP) Principles

1. *Encapsulation: Bundling data (private fields) and methods together, exposing access via public getters/setters.*
2. *Abstraction: Hiding complex implementation details using abstract classes and interfaces.*
3. *Inheritance: Reusing code where child subclass derives attributes and behaviors from a parent base class.*
4. *Polymorphism: Ability of objects to take multiple forms through method overriding (runtime) and method overloading (compile-time).*

Q5: RSA Public Key Cryptography & Asymmetric Security

Select primes p, q . Compute modulus $n = p * q$ and totient $\phi(n) = (p-1) * (q-1)$.

Choose public key exponent e coprime to $\phi(n)$. Compute private key d such that $e * d = 1 \pmod{\phi(n)}$.

- *Encryption: Ciphertext $c = m^e \pmod{n}$ using Public Key (e, n) .*
- *Decryption: Plaintext message $m = c^d \pmod{n}$ using Private Key (d, n) .*

Asymmetric mechanics allow sender to encrypt with public key without knowing private secret.

